```
#2

hello = identity (name) -> "Hello #{name}"
# [Function]

hello 'Scruffy'
# 'Hello Scruffy'
```

In the listing that follows, you can see a tiny command-line CoffeeScript program that can execute a .scruffycoffee file using this technique. A .scruffycoffee file is simply a CoffeeScript program that also supports Scruffy's desired λ syntax.

Listing 8.6. ScruffyCoffee with `eval` and regular expressions

```
fs = require 'fs'
coffee = require 'coffee-script'

evalScruffyCoffeeFile = (fileName) ->
  fs.readFile fileName, 'utf-8',(err, source) ->
    coffeeCode = source.replace /λ([a-zA-Z]+)[.]([a-zA-Z]+)/g,'($1) -> $2'
    coffee.eval coffeeCode

fileName = process.argv[2]
unless fileName
  console.log 'No file specified'
  process.exit()
evalScruffyCoffeeFile fileName
```

Manipulating strings often means heavy use of regular expressions. They're powerful and succinct, but for complicated tasks they can become difficult (or impenetrable) for others to read.

Imagine the possibilities when you can evaluate arbitrary snippets of code. Now, stop imagining the possibilities and consider instead your responsibility to write comprehensible programs. Still, the presence of `eval` means you can metaprogram to your heart's content with just a bit of string interpolation. Apart from the dangers, CoffeeScript isn't very well suited for this.

The regular expression and `eval` solution aren't really workable, and the program in listing 8.6 is very limited due to the simplicity of the regular expression. As you try to make the regular expression more complete, you'll realize that you're essentially just poorly reimplementing the CoffeeScript tokenizer for no good reason. Instead, you'll be better off modifying the token stream or the AST.